Reading Assignment – Lab 03 Section 16

Vu Duy Quang

April 17, 2026

1. What are the advantages of Polymorphism?

Polymorphism provides several important advantages in object-oriented programming:

- **Code reusability and flexibility**: The same code can work with objects of different classes that share a common parent (e.g. `Media`, `Disc`). We can write one method like `playMedia(Playable p)` and it works for both `DigitalVideoDisc` and `CompactDisc` without changing the code.
- **Extensibility**: It is easy to add new types of media (e.g. a future `Magazine` class) without modifying existing code.
- **Simpler and cleaner code**: We can treat different objects uniformly through a common interface or superclass (e.g. a list of `Playable` objects or `Media` objects).
- **Loose coupling**: Classes depend on abstractions (interfaces or abstract classes) rather than concrete implementations, making the system easier to maintain and test.

2. How is Inheritance useful to achieve Polymorphism in Java?

Inheritance is the main mechanism that enables **runtime polymorphism** (dynamic method dispatch) in Java.

- A child class inherits the methods of its parent class (e.g. `DigitalVideoDisc` and `CompactDisc` both inherit from `Disc` which inherits from `Media`).
- The child classes can **override** methods of the parent (e.g. `play()` method).
- When we store objects in a parent-type reference (upcasting), Java decides **at runtime** which version of the overridden method to call based on the actual object type.

Example from our AIMS project:

```
Media media = new DigitalVideoDisc(...);   // upcasting
media.play();   // calls DigitalVideoDisc's play() at runtime
```

Without inheritance, we would not be able to achieve this "one interface, many implementations" behavior.

Aspect	Inheritance	Polymorphism
Definition	Mechanism where one class acquires the properties and behaviors of another class ("is-a" relationship)	Ability of an object to take on many forms (one name, many behaviors)
Purpose	Code reuse and establishing class hierarchy	Flexibility and treating different objects uniformly
Relationship	Parent-child relationship	"Many forms" of the same method
Keywords / Features	`extends`, `super`	Method overriding + method overloading + interfaces
When it happens	At design time (compile time)	Can be compile-time (overloading) or runtime (overriding)
Example in AIMS	`DigitalVideoDisc extends Disc`	`dvd.play()` and `cd.play()` behave differently even though both are called via `Playable` reference

3. What are the differences between Polymorphism and Inheritance in Java?

In short: Inheritance is the tool/mechanism. Polymorphism is the powerful behavior/feature we gain by using that tool .